QuoteFlow

Architecture & Completion Status — the pictorial record

Field	Value
Document	Architecture & Completion Status (diagram pack)
Document ID	claude_V001_MoPr04-quoteflow_hubspot_architecture_20260920_1813
Product	QuoteFlow — CPQ for the HubSpot App Marketplace (portfolio ref 003-MoPr04)
Version	v001
Date	20 September 2026, 18:13
Author	Claude for Senthil/KP
Status	Released
Code baseline	Tree 43de1cac3aa5a9aa · 70 source files · 17 migrations · 793 tests / 45 suites at 100% coverage floor
Companion documents	claude_v028 sprint tracker · claude_v036 product workbook · claude_V001 manual test pack · quoteflow_extreview2_codebase_20260920_1020.zip
Classification	Internal. Contains an explicit record of what is NOT built; not for external distribution.

Why this document exists

Two external reviews in a week both reached the same conclusion by different routes: the build looked far more finished than it was, because nothing distinguished code that had been proved from code that had merely been written. This pack is the corrective. Every element of the product is drawn once, and coloured by the strength of the evidence behind it rather than by how complete it feels.

There are exactly three colours, and the difference between them is the whole point. Green means an automated test demonstrates the behaviour against a real engine or a real payload shape. Amber means the code exists and has never met the real third party. Red means it is not built, and could not be built here — it needs an account, a hosted environment, a purchase or a customer.

Version control

This document follows the same lineage discipline as the sprint tracker and the product workbook: one row per issued version, no row edited after release, and a PLANNED row recorded before work starts so an abandoned attempt leaves a trace.

Version	Date	Author	Change	Status
v001	2026-09-20 18:13	Claude for Senthil/KP	First issue. Two figures drawn from the v028/v036 baseline: (1) the architecture end to end, colour-coded by evidence strength; (2) the quote journey with the three gaps that decide whether any of it matters. Cover, version control, how to read, completed register, not-completed register with reasons and owners, and the evidence-class rule.	Released
v002	— (planned)	—	Will be issued when the first red box turns: a HubSpot developer account (R-Q15) would move the entire app-surface band from amber to green and is the single highest-value change to this picture.	Planned

Change control

A figure may not be recoloured without a corresponding change in the codebase and its evidence register (`src/services/evidence.ts`). The register is enforced by a test and by CI, so a box that turns green here without turning green there will fail the build — which is the intended direction of causality.

How to read this pack

Page	What it is	How to use it
1 — Cover	Identity, baseline and classification of this issue.	Check the code baseline hash matches the zip you are reviewing before trusting any other page.
2 — Version control	Lineage and change-control rule.	Add a row; never edit one. Record the PLANNED row before starting work.
3 — Figure 1: architecture	The whole product in six bands, from HubSpot down to storage.	Read top to bottom: this is the path a single request takes. Read the colours across: this is how much of that path is proved.
4 — Figure 2: journey and gaps	What a rep would actually do, and what has never been proved beneath each step.	Use it in a status conversation. The bottom row is the honest agenda.
5 — What is complete	Every completed capability with the evidence that closes it.	This is what may be demonstrated. Each row names the test that proves it.
6 — What is not complete	Every gap with its reason, its owner and what would unblock it.	This is the decision list. Nothing on it is unblocked by writing more code.
7 — The evidence rule	The claims register and why a coverage number is not a quality argument.	Read before writing any listing, trust-page or investor copy.

Figure 1 — What the product is, and how far it is actually built

Read down for the path of a request: HubSpot → the app card → the front door → the engines → durability → storage. Read across for evidence. Bands 3 to 6 are almost entirely green because they are testable here. Band 2 is amber because it can only be proved by installing it, and band 1 contains the red box that gates everything.

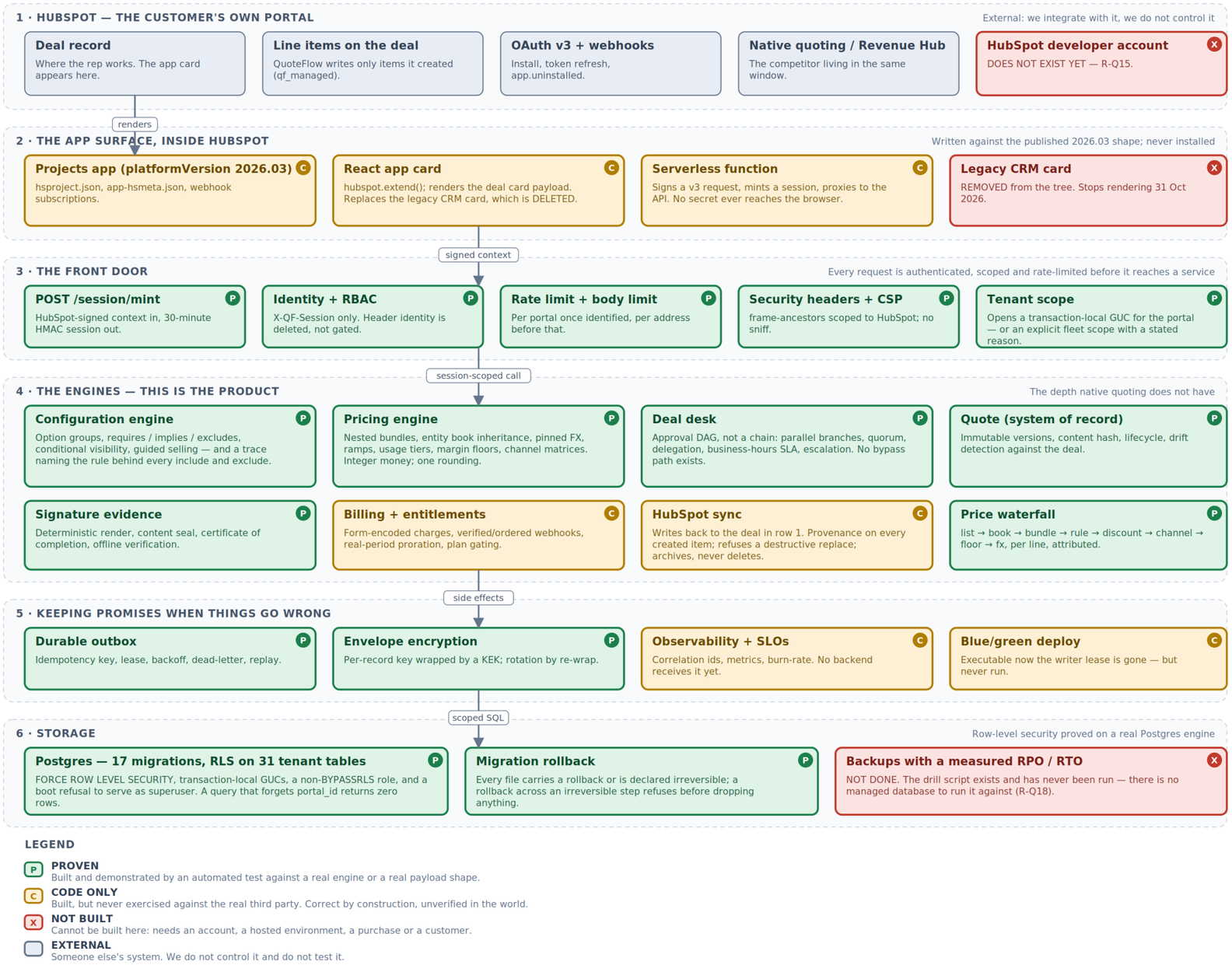
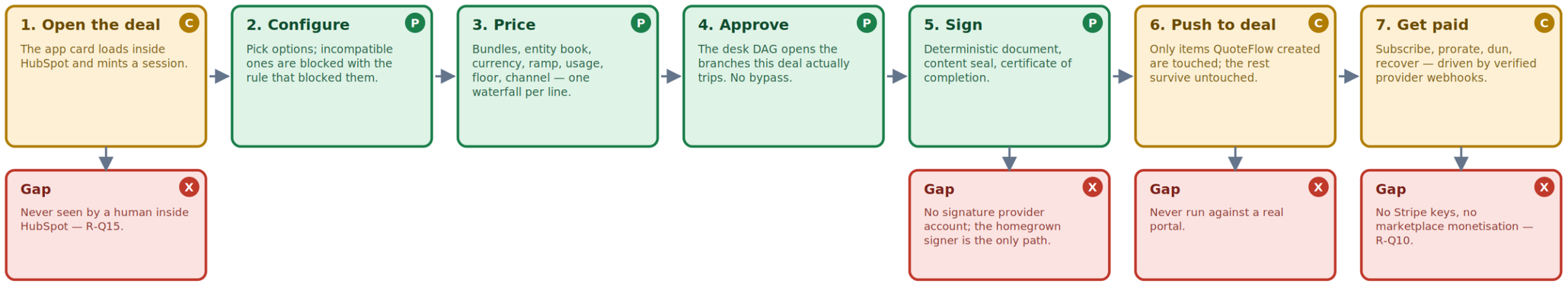


Figure 2 — The quote journey, and the three gaps that decide whether it matters

The top row is the product as a rep would experience it. The row beneath is what has never been proved at that step. The three panels below are the gaps that no further engineering will close.

QuoteFlow — the quote journey, and the three gaps that decide whether it matters

The top row is what a rep would do. The row beneath it is what has never been proved.



WHAT DECIDES WHETHER ANY OF THIS MATTERS

The platform gap X

Nothing has ever run inside HubSpot. The Projects app and the app card are written against the published 2026.03 developer-platform shape and have never been installed, rendered or accepted. Until a developer account exists, every green box above is a statement about our own tests rather than about the product.

The commercial gap X

No listing, no design partner, no paying customer, no churn data. The configuration scenarios come from limitations HubSpot documents, not from a prospect who asked. The depth may be real and still be depth in the wrong direction.

The operational gap X

p95 under load, a measured RPO and RTO, a deploy with zero dropped requests, and a clean penetration test are all absent. Two harness scripts are written and have never been run; the other two are purchases, not sprints.

THE RULE THAT KEEPS THIS HONEST

A claim may not be stated above the evidence that backs it. P

Every test file declares the class of evidence it produces (CODE, CONTRACT, LIVE, SECURITY, PERF, OPS, COMMERCIAL). Every published claim carries the classes it requires and the classes it has. CI fails the build when a test file is untagged, when a class a shipped feature claims is empty, or when a gap has no named owner. 8 of 15 claims are currently supported; 7 are blocked, and each names what would unblock it. The 100% coverage gate is a REGRESSION FLOOR — it stops behaviour disappearing unnoticed, and says nothing about whether the behaviour is right.

What is complete

Fifteen capabilities are complete in the strongest sense available without leaving this machine: an automated test demonstrates the behaviour, and the test names the thing it proves. Each row below is a green box in Figure 1.

Capability	What it does	Evidence	Proved by	Closes
Tenant isolation	Row-level security on 31 tenant tables: transaction-local GUCs, FORCE RLS, a non-BYPASSRLS role, and a boot refusal to serve as superuser.	PROVEN	tests/tenantRls.test.ts on PGlite — a query that omits portal_id returns zero rows on a real Postgres engine.	PXR-003
Durable secrets	OAuth tokens envelope-encrypted (AES-256-GCM, per-record key wrapped by a KEK) before they reach the database; rotation by re-wrap.	PROVEN	tests/storeContract2.test.ts, envelope seal/open/rewrap and key rotation.	PXR-002
Durable side effects	Outbox with idempotency keys, leased claims, exponential backoff, dead-letter and replay. Nothing is lost when a process dies.	PROVEN	tests/outbox.test.ts.	PXR-002, PXR-016
Multi-instance operation	Every store is async and cache-free; the writer lease is retired, so two instances share one database and see each other’s writes.	PROVEN	tests/pgBoot.test.ts — two wired instances over one database, no reload step.	PXR-005
Production identity	Sessions minted at POST /session/mint from a HubSpot-signed context; the unsigned header path is deleted, not gated.	PROVEN	tests/s3api.test.ts PS16 block — forged, stale, malformed and expired all refused.	PXR-013
Customer data safety	A push touches only line items QuoteFlow created; a destructive replace is refused; items are archived, never deleted.	PROVEN	tests/extreview2.test.ts PXR-004 block.	PXR-004
Quote as system of record	Immutable versions with content hashes, a lifecycle, an explicit field-ownership table, and drift detection against the deal.	PROVEN	tests/quotes.test.ts.	PXR-008
Signature evidence	Deterministic rendering, a content seal over the exact bytes sent, a certificate of completion, and offline verification.	PROVEN	tests/signatureEvidence.test.ts — altering one byte fails verification.	PXR-007
Configuration engine	Option groups, requires/implies/excludes, conditional visibility, guided selling, a documented tie-break, and a trace naming the rule behind every decision. Contradictory or circular rule sets are refused at save time.	PROVEN	tests/configuration.test.ts — five scenarios native quoting cannot express; tests/cpqApi.test.ts over HTTP on both backends.	PXR-009
Pricing engine	Nested bundles, entity book inheritance, pinned FX with per-currency exponent and rounding, ramps, usage commit and overage, cost-plus and margin floors, channel matrices, and a full per-line waterfall.	PROVEN	tests/pricingDepth.test.ts — three currencies exact to the minor unit; February and March prorate differently.	PXR-010 (quoting half)
Deal desk	Approval DAG with parallel branches, quorum, delegation, business-hours SLA, escalation ladders, recall/resubmit with version-aware invalidation, and an append-only audit with no bypass path.	PROVEN	tests/dealDesk.test.ts — eight distinct bypass attempts, each denied AND recorded.	PXR-009
Migration rollback	Every file carries a rollback or is declared irreversible; a rollback across an irreversible step refuses before dropping anything.	PROVEN	tests/operations.test.ts — apply 017, roll back to 013, apply forward again on a real engine.	PS30-04
Graceful degradation	Postgres, HubSpot and the mail provider each killed: retryable errors, a failed sync recorded rather than thrown away, mail dead-lettered rather than dropped.	PROVEN	tests/operations.test.ts chaos block.	PS30-05
Posture declarations	Data-flow encryption stated per store, three residency regions declared available, and a subprocessor list separating in-use from chosen.	PROVEN	tests/operations.test.ts, review2.test.ts — a store cannot claim more than the code does.	PXR-006, PXR-015
The evidence system	Evidence-class tags on every test file, a claims register with required versus held classes, a CI gate, and the matrix carried in the release manifest.	PROVEN	tests/evidence.test.ts; .github/workflows/ci.yml; release-manifest.json.	PXR-011

What is not complete, and why

Every row below is red or amber in the figures. The column that matters is the third one: in every case the reason is an account, an environment, a purchase or a customer — not an unwritten module. No amount of further engineering closes any of these.

Item	State	Why it is not complete	Owner / what unblocks it	Claim	Consequence of leaving it
Running inside HubSpot	NOT BUILT	A HubSpot developer account does not exist. The Projects app, the app card and the serverless proxy are written against the published 2026.03 developer-platform shape and have never been uploaded, installed, rendered or accepted.	Senthil — create a HubSpot developer account (R-Q15)	CL-01	Highest. Until this turns, the whole product is a set of statements about our own tests. It also gates the LIVE evidence class for every other feature.
Billing against a real provider	CODE ONLY	Stripe form-encoding, the marketplace rail, webhook verification, replay tolerance, out-of-order handling and proration are all written and unit-tested. No Stripe key and no HubSpot monetisation access exist, so no payload has ever been accepted by a provider and no webhook has ever arrived.	Senthil — Stripe test-mode keys; apply for marketplace monetisation (R-Q10)	CL-09	High. A billing bug is a revenue bug, and CODE evidence does not catch a field name the provider rejects.
p95 under load	NOT BUILT	There is no deployed environment to measure. scripts/loadtest.js is written and has never been run; running it against localhost would prove localhost is fast, which is not the claim.	Senthil — a hosting target with two instances behind a load balancer (R-Q17)	CL-10	Medium. The architecture supports it, but a performance number that has not been measured is not a number.
Measured RPO and RTO	NOT BUILT	No managed Postgres with point-in-time recovery exists. scripts/restore-drill.sh restores, proves the restored database serves tenant-scoped reads, and prints both figures — once there is something to run it against.	Senthil — managed Postgres with PITR (R-Q18)	CL-11	Medium-high for an enterprise buyer: this is question three on every security questionnaire.
A deploy with zero dropped requests	CODE ONLY	The writer lease that made blue/green impossible is gone, and the workflow is blue/green again with a tripwire that fails the build if the lease reappears. No deploy has ever been run.	Follows the hosting decision (R-Q17)	CL-12	Medium. The blocker was removed; the proof was not obtained.
External penetration test	NOT BUILT	ABUSE_CASES exists as a scope document and the adversarial tests cover forged sessions, cross-tenant reads and every approval bypass. An independent test is a purchase, not a sprint.	Senthil — engage a testing firm (R-Q12)	CL-13	Medium. Our own adversarial tests are SECURITY evidence; they are not an independent assessment.
SOC 2 readiness	NOT BUILT	An assessment, not an engineering task. The posture declarations, audit trail and data-flow map are the inputs it would consume.	Senthil — commission an assessment	—	Low until there is a customer who asks for it; then it becomes a gate.
Paying customers	NOT BUILT	No listing, no design partner, no revenue, no churn data. The financial model remains assumption-led and is flagged as such on its own sheet.	Senthil — PS13 positioning decision, then PS32 validation	CL-14	Decisive. This is the only gap on the list that determines whether the others are worth closing.
Invoice tax depth	PARTIAL	Quoting is fully multi-currency with per-currency rounding and multi-entity books. INVOICING still applies a single flat tax rate per portal, so the PXR-010 finding is half-closed and is recorded as half-closed rather than counted as done.	Next build sprint — a jurisdiction table and per-line tax codes (R-Q20)	—	Low now, blocking for any EU or Indian GST customer.
Design-partner validation of the depth	NOT BUILT	The five configuration scenarios come from limitations HubSpot documents, not from a prospect who asked for them. The engine may be deep in the wrong direction.	Senthil — PS13/PS32 (R-Q19)	—	High and quiet: this is the failure mode that looks like success right up to launch.

The rule that keeps the picture honest

A claim may not be stated above the evidence class that backs it. This is enforced in code, not by convention: every test file declares the classes it produces, every published claim carries the classes it requires and the classes it holds, and the build fails when a file is untagged, when a class a shipped feature claims is empty, or when a gap has no named owner.

Evidence class	Test files	What it proves — and what it does not
CODE	45	The code does what the code says, against our own doubles. The cheapest class, and the only one a coverage gate speaks to.
CONTRACT	2	A third party’s API shape is honoured and its real error shapes are handled. Recorded real fixtures count; invented ones do not.
LIVE	0	EMPTY. Nothing has run end to end against a real HubSpot account watched by a human.
SECURITY	8	An adversarial test attempted the attack and failed: forged sessions, cross-tenant reads, every approval bypass, tampered packages.
PERF	0	EMPTY. No figure has been measured under stated load.
OPS	4	An operation was actually run and measured — here, migration rollback and induced-failure degradation. A restore and a deploy are still missing.
COMMERCIAL	0	EMPTY. Nobody outside this build has paid or committed to pay.

Claims register — the state today

8 of 15 claims are supported and may be published. 7 are blocked and may not appear on a listing, a trust page, a README or a pitch until their gap closes.

Blocked claim	Missing class	Owner / what unblocks it
CL-01 — Quotes are configured, priced and approved inside HubSpot, on the deal record.	LIVE	R-Q15 — needs a HubSpot developer account before any app card can be installed
CL-09 — Billing survives replayed and out-of-order provider webhooks.	CONTRACT	R-Q10 — needs Stripe test-mode keys and HubSpot monetisation access
CL-10 — Two instances serve p95 under 500ms at a stated concurrency.	PERF	PS30-01 — needs a hosted environment to load-test against
CL-11 — Backups restore with a measured RPO and RTO.	OPS	PS30-03 — needs a managed Postgres with point-in-time recovery
CL-12 — A deploy happens with zero dropped requests.	OPS	PS30-02 — the lease is gone and the workflow is blue/green, but no deploy has been run
CL-13 — An external penetration test found no critical or high issues.	SECURITY	PS30-06 — a pen test is a purchase, not a sprint
CL-14 — Customers pay for QuoteFlow.	COMMERCIAL	PS32 — no listing, no design partners, no revenue

On the coverage number

The suite stands at 793 tests across 45 suites with a 100% statement, branch, function and line floor. That floor is a **regression floor**. It stops behaviour disappearing unnoticed between releases. It is not a quality argument, it never was, and presenting it as one is precisely what both external reviews objected to. The figures in this pack are coloured by evidence class for that reason: the colour survives the coverage number being perfect, which it already is.

The one-line summary

The engineering objections are answered. The platform, commercial and operational objections are not, and no further code will answer them.